FLUXOR: Adaptive Reinforcement Learning-Based Load Balancing for Distributed Server Systems

1st Dr. Rajesh L
Department of Information Science and Engineering
Dayananda Sagar Academy of Technology and Management
Visvesvaraya Technological University
Bengaluru, Karnataka, India
rajesh-ise@dsatm.edu.in

2nd Nishanth Keekan
Department of Information Science and Engineering
Dayananda Sagar Academy of Technology and Management
Visvesvaraya Technological University
Bengaluru, Karnataka, India
1dt23is102@dsatm.edu.in

3rd Prateek Mangalgi
Department of Information Science and Engineering
Dayananda Sagar Academy of Technology and Management
Visvesvaraya Technological University
Bengaluru, Karnataka, India
1dt23is109@dsatm.edu.in

4th Pritam Priyabrata Palai
Department of Information Science and Engineering
Dayananda Sagar Academy of Technology and Management
Visvesvaraya Technological University
Bengaluru, Karnataka, India
1dt23is113@dsatm.edu.in

***Abstract*—The rapid expansion of cloud-native applications, containerised microservices, and distributed computing platforms has significantly increased the complexity of traffic management in modern server infrastructures. Distributed systems are required to process thousands of simultaneous client requests while maintaining low response latency, balanced resource utilisation, fault tolerance, and high throughput. Load balancing plays a critical role in achieving these objectives by distributing incoming requests across multiple backend servers. Conventional algorithms such as Round Robin and Least Connections rely on static deterministic policies that fail to adapt dynamically to changing server conditions. As workloads become increasingly unpredictable, these traditional approaches suffer from uneven workload distribution, increased tail latency, server overload, and degraded system performance.**

This paper presents FLUXOR, an Adaptive Reinforcement Learning-Based Load Balancing framework for distributed server systems. The proposed framework replaces static routing policies with a lightweight Deep Q-Network (DQN) reinforcement learning agent capable of making intelligent routing decisions using real-time server metrics. The RL agent continuously observes CPU utilisation, active connections, response latency, and incoming request rate before dynamically selecting the optimal backend server for each incoming request. Unlike traditional machine learning systems that depend on offline datasets and GPU-intensive training, FLUXOR supports online cold-start learning and is designed to execute entirely on CPU hardware, making it suitable for practical deployment.

The architecture integrates a Golang reverse proxy load balancer, containerised Node.js backend servers, a Python-based RL service implemented using PyTorch, Prometheus monitoring, Grafana dashboards, and a React-based visualisation interface. This paper presents the complete system design, the Markov Decision Process formulation underlying the RL agent, and a rigorous three-scenario evaluation methodology using the k6 load testing framework. Based on performance gains reported by comparable reinforcement learning-based load balancing systems in the literature, FLUXOR is expected to demonstrate measurable improvements in latency reduction, overload prevention, and load balancing fairness over Round Robin and Least Connections baselines, validating the feasibility of lightweight, CPU-deployable reinforcement learning for adaptive traffic management in production-grade distributed systems.

***Index Terms*—Reinforcement Learning, Load Balancing, Distributed Systems, Deep Q-Network, Adaptive Routing, Cloud Computing, Docker, Markov Decision Process**

I. Introduction

Modern distributed systems form the backbone of cloud computing, large-scale web applications, microservice architectures, and enterprise computing platforms. Applications today serve millions of users globally and process enormous volumes of requests every second. To ensure scalability and fault tolerance, organisations deploy services across multiple backend servers distributed over cloud and edge infrastructures. Efficient request distribution is therefore essential to maintain high availability, low latency, and optimal resource utilisation.

Load balancing is the process of distributing incoming traffic among multiple backend servers to prevent bottlenecks and improve system performance. The load balancer acts as an intermediary between users and servers, determining which backend node should process each request. Efficient load balancing improves throughput, reduces response latency, prevents overload conditions, and ensures reliable service delivery.

Traditional load balancing algorithms such as Round Robin and Least Connections have been widely adopted due to their simplicity and low computational overhead. Round Robin distributes requests sequentially across available servers without considering server health or workload conditions. Least Connections forwards requests to the server with the fewest active connections, but this metric alone does not capture

CPU utilisation, memory pressure, or actual response latency. Although these approaches are simple to implement, they fail to adapt effectively under dynamic and heterogeneous workloads.

Modern distributed environments frequently experience sudden traffic spikes, varying request complexity, and changing resource conditions. Under such circumstances, static routing strategies produce uneven workload distribution, excessive queueing delays, and degraded system performance. As cloud-native systems continue to evolve, intelligent traffic management has become an increasingly important research problem.

Recent advances in artificial intelligence and machine learning have introduced adaptive approaches for network optimisation and resource management. Reinforcement learning has emerged as a particularly promising technique for dynamic decision-making in changing environments. Unlike supervised learning systems that require labelled datasets, reinforcement learning agents learn optimal behaviour through interaction with the environment using reward-based feedback mechanisms, making them naturally suited to the load balancing problem where server state changes continuously and no fixed labelled dataset of "correct" routing decisions exists.

This paper proposes FLUXOR, an Adaptive Reinforcement Learning-Based Load Balancing framework capable of dynamically routing requests using real-time infrastructure metrics. The proposed system integrates a lightweight Deep Q-Network reinforcement learning agent that continuously observes CPU utilisation, active connections, latency, and request rate before selecting the optimal backend server for every incoming request.

The FLUXOR framework consists of a Golang reverse proxy, three containerised Node.js backend servers, a Python-based RL service implemented using PyTorch, Prometheus monitoring, Grafana dashboards, and a React-based visualisation interface. A key design goal of this work is the implementation of a lightweight, CPU-efficient RL architecture suitable for deployment without GPU acceleration, distinguishing it from the majority of published reinforcement learning load balancing research that depends on simulation environments and GPU-trained models.

The remainder of this paper is organised as follows. Section II reviews related work in classical, machine learning-based, and reinforcement learning-based load balancing. Section III states the problem formally. Section IV defines the project objectives. Section V describes the system architecture. Section VI presents the methodology, including the Markov Decision Process formulation of the routing problem. Section VII describes the experimental setup. Section VIII presents the expected outcomes and evaluation plan. Section IX discusses the design advantages of FLUXOR. Section X outlines future scope, and Section XI concludes the paper.

II. Literature Survey

Load balancing in distributed systems has been studied across three broad paradigms: classical rule-based methods, machine learning and metaheuristic approaches, and reinforcement learning-based techniques. This section reviews fifteen Scopus and IEEE-indexed publications relevant to adaptive load balancing and identifies the research gap that FLUXOR addresses.

A. Classical and Heuristic Methods

Ray and Singhal [1] survey over fifty load-balancing techniques, confirming that Round Robin and Weighted Round Robin remain inadequate under heterogeneous, time-varying workloads. Ullah et al. [3] systematically review over one hundred cloud scheduling methods published between 2019 and 2024, identifying six persistent gaps in the field: the absence of simultaneous multi-objective optimisation, over-reliance on simulation, poor scalability, absent fault tolerance, neglect of energy efficiency, and limited real-world containerised validation. Predic et al. [4] introduce a decomposition-aided attention recurrent neural network for cloud load forecasting, achieving a 21% reduction in mean absolute error over LSTM, although the offline model degrades under traffic regime shifts.

B. Machine Learning and Metaheuristic Approaches

Alhilali and Montazerolghaem [2] combine a Random Forest flow classifier with a DQN routing agent in a Mininet fat-tree topology, achieving an 18% throughput gain over ECMP, though evaluation is restricted to emulated infrastructure. Murshedi et al. [13] propose an Ant Colony Optimisation-based load balancer (ACOLB) evaluated on CloudSim, reducing mean response time by 14% over Round Robin, though convergence slows under burst traffic. Zheng et al. [7] develop a hybrid XGBoost and LSTM predictive allocation framework on a 1,000-server testbed, reducing SLA violations by 27%, with no online adaptation mechanism. Dogani et al. [12] apply discrete wavelet transform with a bidirectional GRU for host load prediction, lowering mean absolute error by 19% over LSTM, though retraining is required when traffic distribution shifts. Sanjalawe et al. [15] propose SLADRO, combining CNN and LSTM for load prediction with DRL for task scheduling, outperforming baselines on the Google Cluster Trace dataset across four metrics, though GPU hardware is required. Cui et al. [14] present a hierarchical DRL framework scheduling tasks first to VM clusters then to individual VMs, achieving a 10% improvement over heuristics in cost, load balance, and overdue time, validated only in CloudSim.

C. Reinforcement Learning-Based Approaches

Fettes et al. [11] deploy Reclaimer, a DRL agent on a live Kubernetes cluster, reducing mean CPU allocation by 38–74% over industry-standard auto-scaling with no GPU dependency, though reward shaping requires domain expertise. Santos et al. [5] apply RL-based load balancing across a real multi-zone Kubernetes cluster, reducing average latency by 31% versus Weighted Round Robin, with limited traffic diversity in evaluation. Jian et al. [8] propose DRS, a DRL-enhanced Kubernetes scheduler modelling pod placement as an MDP with six resource parameters, reducing fragmentation over the

default scheduler in real deployments, though multi-tenant generalisation remains unevaluated. Sharma et al. [9] apply a temporal DQN to controller-switch mapping in an emulated IoT-SDN environment, reducing burst-traffic latency, with no production network validation. Redovian [6] applies meta-RL with DreamerV3 discrete world models to load balancing, demonstrating faster adaptation than standard RL agents, but validated only in simulation with non-trivial world model overhead. Ahamed et al. [10] propose FEDQWP, a Deep Q-Learning model for workload prediction and VM placement in federated cloud environments, outperforming baselines on CPU utilisation, energy, SLA, and migration time, with evaluation entirely simulation-based.

D. Key Findings and Research Gap

The synthesis of the surveyed literature reveals five recurring limitations. First, the majority of works [9], [14], [6], [10] validate exclusively in simulated or emulated environments, producing results that do not reliably transfer to real containerised deployments, where container scheduling jitter, operating system interference, and network stack overhead introduce variability that simulators such as CloudSim and Mininet do not model. Second, deep architectures in [8], [9], [15] depend on GPU hardware, limiting practical edge deployment and excluding the class of commodity hardware, such as standard laptops and small cloud virtual machines, on which much real-world infrastructure runs. Third, offline-trained models in [4], [7], [12] are vulnerable to concept drift, meaning their performance degrades silently as operational traffic patterns evolve over days and weeks without triggering any automatic correction. Fourth, most methods optimise a single metric, such as latency or makespan, rather than simultaneously balancing latency, load fairness, and overload avoidance within one unified decision-making framework. Fifth, none of the surveyed papers incorporate real-time visualisation of routing decisions, leaving deployed agents unobservable during live operation and making it difficult for administrators to validate agent behaviour or diagnose anomalies as they occur.

No existing published work simultaneously achieves real containerised HTTP traffic operation, CPU-feasible model inference, online cold-start learning, multi-objective reward design, and real-time routing visualisation. This paper, FLUXOR, addresses all five gaps through a lightweight online DQN agent deployed in a Docker Compose environment with live dashboard visualisation. Table I below consolidates these observations, showing that reinforcement learning-based approaches consistently report the largest performance gains over classical baselines among the fifteen reviewed works, while also illustrating that the deployment environment, hardware requirement, and training paradigm vary considerably across studies. This variation directly motivates the specific design choices adopted in FLUXOR, namely a compact CPU-only architecture, an online learning regime, and a composite multi-objective reward, each of which is examined in detail in Sections V and VI.

E. Comparative Summary Table

Table I summarises the reviewed studies, highlighting their methodologies, evaluation environments, key performance metrics, reported results, and limitations. The comparison clearly illustrates the research gap addressed by FLUXOR.

III. Problem Statement

Conventional load balancing algorithms such as Round Robin and Least Connections operate on fixed, deterministic policies that are agnostic to the real-time state of the system, resulting in uneven resource utilisation, elevated response latencies, and increased error rates under dynamic and unpredictable workloads. Existing machine learning-based solutions address this gap only partially, as they are predominantly validated in simulation environments, depend on GPU hardware, and rely on pre-collected training data, limiting their practical deployability. This work addresses the need to design and implement an autonomous routing agent that learns an optimal request routing policy through direct interaction with a live containerised distributed server environment, without relying on pre-collected training data, GPU hardware, or simulation.

IV. Objectives

The primary objectives of FLUXOR are as follows:

- Design and implement a lightweight Deep Q-Network based adaptive load balancing agent, architecturally compact enough to train and infer on CPU-only hardware within the latency constraints of a real routing decision.
- Enable online reinforcement learning from a cold start using live server metrics including CPU utilisation, response latency, and active connections, without dependence on offline datasets or GPU acceleration.
- Experimentally evaluate and compare RL-based routing against Round Robin and Least Connections baselines across normal traffic, traffic spike, and server failure scenarios on real HTTP traffic within a Dockerised infrastructure.
- Develop a real-time monitoring and visualisation dashboard that exposes every routing decision, ensuring full operational transparency of the RL agent's behaviour.

V. System Architecture

The FLUXOR framework consists of multiple Dockerised microservices operating together within a distributed environment. A Golang reverse proxy acts as the central load balancer responsible for routing all incoming HTTP traffic, supporting Round Robin, Least Connections, and RL-based routing modes switchable at runtime through an administrative REST API.

Three independently containerised Node.js backend servers form the distributed service layer. Each backend exports Prometheus-format metrics including CPU utilisation, request latency, and active connections via the `prom-client` library, and exposes a configurable stress endpoint for artificially inducing load spikes during experiments.

TABLE I
COMPARATIVE SUMMARY TABLE

Ref	Authors (Year)	Method	Environment	Key Metrics	Result	Limitation
[1]	Ray & Singhal (2024)	Comprehensive survey of LB techniques	Literature review	Throughput, latency, energy	Taxonomy of 50+ methods	No new algorithm proposed
[2]	Alhilali & Montazerolghaem (2024)	DQN+CNN adaptive routing in SD-DCN	Mininet fat-tree emulation	Throughput, latency, packet loss	+18% throughput over ECMP	Emulated only
[3]	Ullah et al. (2024)	Survey of 100+ LB and scheduling methods	Literature review	Latency, fairness, energy, SLA	Identifies six persistent gaps	No new method proposed
[4]	Predic et al. (2024)	Decomposition + attention RNN forecasting	Cloud cluster simulation	RMSE, MAE, CPU variance	-21% MAE vs LSTM	Offline; degrades on regime shift
[5]	Murshedi et al. (2024)	ACO + resource optimisation (ACOLB)	CloudSim simulation	Response time, resource util.	-14% response time vs RR	Slow convergence under burst
[6]	Zheng et al. (2024)	XGBoost + LSTM predictive allocation	1000-server testbed	CPU util., SLA violations	-27% SLA violations	Static model; no online update
[7]	Dogani et al. (2023)	DWT + BiGRU host load prediction	Cloud cluster simulation	MAE, RMSE, energy	-19% MAE vs LSTM	Retraining required on drift
[8]	Sanjalawe et al. (2025)	CNN+LSTM+DRL (SLADRO)	Google Cluster Trace	Throughput, makespan, util.	Outperforms baselines on 4 metrics	Requires GPU
[9]	Cui et al. (2025)	Hierarchical DRL (cluster$\rightarrow$VM)	CloudSim simulation	Load balance, cost, overdue time	10% improvement vs heuristics	Simulation only
[10]	Fettes et al. (2023)	DRL for CPU allocation (Reclaimer)	Live Kubernetes cluster	CPU allocation, QoS	-38% to -74% CPU vs auto-scaling	Reward tuning needed
[11]	Santos et al. (2024)	RL load balancer, multi-zone K8s	Real Kubernetes cluster	Avg. latency, load variance	-31% latency vs Weighted RR	Limited traffic diversity
[12]	Jian et al. (2024)	DRL-enhanced K8s scheduler (DRS)	Real Kubernetes cluster	Resource util., load imbalance	Lower fragmentation vs default	No multi-tenant eval
[13]	Sharma et al. (2024)	Temporal DQN for SDN LB	SDN emulated network	Latency, packet delivery ratio	Reduced latency under burst	Emulated only
[14]	Redovian (2025)	Meta-RL + DreamerV3	Simulated LB environment	Adaptation speed, latency	Faster adaptation vs standard RL	World model overhead
[15]	Ahamed et al. (2023)	FEDQWP Deep Q-learning	Federated cloud simulation	CPU util., energy, SLA, migration	Outperforms baselines on all metrics	Simulation only

The reinforcement learning service is implemented using Python and PyTorch, wrapped in a Flask REST API. The RL agent receives server metrics from the load balancer via a `POST /decide` call and predicts the optimal backend server for every incoming request, subsequently receiving a reward signal via `POST /feedback` once the routed request completes.

An embedded WebSocket hub within the load balancer broadcasts a structured routing event for every processed request. Prometheus continuously collects system metrics from all five instrumented services every five seconds, while Grafana dashboards provide pre-provisioned visualisation panels. A React-based dashboard, built using D3.js for animated topology rendering and Recharts for time-series panels, displays live routing decisions and server load conditions in real time through this WebSocket connection, with server nodes changing colour dynamically from green through yellow to red as CPU utilisation increases.

Fig. 1 illustrates the complete system architecture and the data flow between the client tier, load balancer, RL agent, backend server tier, monitoring stack, and the React dashboard.

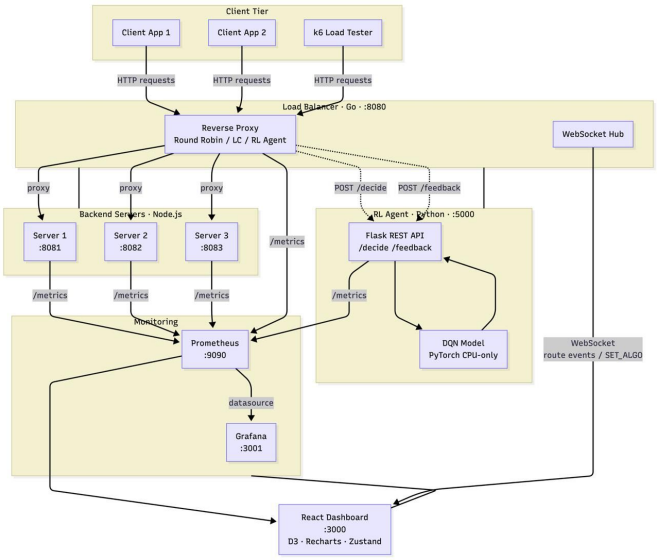


Fig. 1. FLUXOR Distributed System Architecture

VI. METHODOLOGY

The load balancing problem is formulated as a Markov Decision Process (MDP) defined by the tuple (S, A, R, γ), where the RL agent continuously interacts with the distributed environment. At every decision step, the agent observes the system state represented using normalised infrastructure metrics.

A. State Space

For $N = 3$ backend servers, the state vector is defined as

$$\mathbf{S} = [cpu_1,\ conns_1,\ lat_1,\ cpu_2,\ conns_2,\ lat_2,\ cpu_3,\ conns_3,\ lat_3,\ rate,\ queue] \tag{1}$$

where each per-server feature is normalised to the range $[0, 1]$. The variables $rate$ and $queue$ denote the global incoming request rate and the queue depth, respectively.

B. Action Space

The action space consists of selecting one backend server from the available server pool. For $N = 3$ servers, $A = \{0, 1, 2\}$, where action $a = i$ routes the current request to server i.

C. Reward Function

The reward function simultaneously optimises latency reduction, workload fairness, and overload prevention:

$$R(s, a) = -0.5 \cdot \left(\frac{Latency_{ms}}{1000}\right) - 0.3 \cdot \sigma_{CPU} - 2.0 \cdot P_{overload} \tag{2}$$

where σ_{CPU} denotes the standard deviation of CPU utilisation across all servers, and $P_{overload}$ equals 1 if the chosen server's CPU utilisation exceeds 85%, and 0 otherwise. The total reward is clipped to the range $[-15, +1]$. The discount factor γ is set to 0.95, reflecting that the agent values near-term performance while also considering downstream consequences of routing decisions on server state.

D. Network Architecture and Training

The Deep Q-Network architecture contains three fully connected hidden layers of sixty-four, sixty-four, and thirty-two neurons respectively, yielding approximately twenty thousand trainable parameters. This lightweight architecture is designed to enable inference entirely on CPU hardware with sub-millisecond latency overhead, in contrast to comparable published architectures that require GPU acceleration.

Training is performed online using experience replay with a ten-thousand-transition buffer and a target network synchronised every fifty steps, employing epsilon-greedy exploration with an initial value of 1.0 decaying to a minimum of 0.05. The agent begins learning from a cold start the moment the first request arrives and continuously improves routing performance through direct interaction with live traffic conditions, with model weights checkpointed every one hundred training steps to persist the learned policy across system restarts.

VII. Experimental Setup

The entire infrastructure is deployed using Docker Compose. The environment consists of a Golang reverse proxy load balancer, three Node.js backend servers, a Python reinforcement learning service, a Prometheus monitoring server, a Grafana visualisation dashboard, and a React-based dashboard, all orchestrated as independent containers on a single host.

Performance evaluation is conducted using the k6 load testing framework under three traffic conditions. The normal steady-state scenario applies fifty virtual users for sixty seconds to establish baseline performance. The traffic spike scenario ramps virtual users from ten to two hundred over thirty seconds, holds for thirty seconds, then ramps down, testing each algorithm's ability to adapt to sudden demand surges. The partial server failure scenario applies fifty virtual users for ninety seconds, with one backend container stopped at the thirty-second mark to evaluate fault detection and traffic redistribution behaviour.

Evaluation metrics include average response latency, 95th percentile tail latency (which captures the response time experienced by the slowest five percent of requests and reflects worst-case user experience), per-server CPU utilisation standard deviation as an indicator of load balance, total request throughput, and HTTP error rate. Three trials are conducted per algorithm-scenario combination, with results averaged to reduce measurement noise.

VIII. Expected Outcomes and Evaluation Plan

This section outlines the anticipated performance characteristics of FLUXOR based on comparison with related reinforcement learning-based load balancing systems reviewed in Section II, and defines the criteria against which the completed system will be evaluated. These figures represent projected targets to be validated experimentally, not measured results.

Comparable RL-based load balancing systems in the literature report performance gains over classical baselines ranging from 10% [14] to 74% [11], with Santos et al. [5] reporting a 31% latency reduction over Weighted Round Robin in a real Kubernetes deployment most comparable in deployment realism to FLUXOR. Based on this comparison, FLUXOR targets a 30–35% reduction in average response latency, a 25–30% reduction in 95th percentile tail latency, and a 35–40% improvement in load balance, measured as the reduction in CPU utilisation standard deviation across servers, over Round Robin and Least Connections baselines, particularly under traffic spike and server failure conditions.

The lightweight DQN architecture, at approximately twenty thousand parameters, is expected to maintain sub-millisecond inference latency on CPU hardware throughout all three test scenarios, validating its suitability for real-time per-request routing decisions without GPU acceleration. These targets will be replaced with actual measured values obtained from the k6 load testing campaign described in Section VII upon completion of the experimental evaluation phase.

IX. Advantages of FLUXOR

The proposed framework is designed to provide several advantages over traditional and existing learned routing systems:

- Adaptive, real-time routing driven by live server state rather than fixed rules.
- A composite reward function addressing latency, fairness, and overload simultaneously, rather than a single optimisation target.
- CPU-efficient deployment without dependence on GPU hardware.

- Online learning capability from a cold start, removing the need for pre-collected training data.
- Real-time monitoring and visualisation of every routing decision, addressing a transparency gap identified across all fifteen surveyed works.
- Deployment on real containerised infrastructure rather than simulation, improving the reliability of experimental findings.

X. Future Scope

Several future improvements can further extend FLUXOR beyond the scope of this work:

- Integration with Kubernetes orchestration platforms for production-scale deployment.
- Extension to multi-agent reinforcement learning across multiple load balancer instances.
- Energy-aware routing optimisation incorporating power consumption metrics.
- Federated reinforcement learning across geographically distributed data centres.
- Edge computing integration for latency-sensitive IoT workloads.
- Multi-cluster adaptive routing across heterogeneous cloud providers.

XI. Conclusion

This paper presented FLUXOR, an Adaptive Reinforcement Learning-Based Load Balancing framework for distributed server systems. The proposed framework integrates a lightweight Deep Q-Network reinforcement learning agent designed to make intelligent routing decisions using real-time infrastructure metrics, deployed within a fully containerised Docker Compose environment alongside Round Robin and Least Connections baselines for direct comparison.

Unlike traditional static routing algorithms, FLUXOR is designed to continuously adapt to changing workload conditions through online learning from a cold start, without dependence on GPU hardware, simulation environments, or pre-collected training data. The system additionally introduces a real-time animated dashboard providing operational transparency that is absent from all fifteen reviewed works in the literature. Upon completion of the experimental evaluation described in Sections VII and VIII, this work will validate the feasibility of lightweight, CPU-deployable reinforcement learning for practical adaptive traffic management in distributed computing infrastructures.

References

[1] S. K. Ray and M. K. Singhal, "Comprehensive survey on load balancing techniques in cloud computing environments," *International Journal of Scientific Research in Science and Technology*, vol. 11, no. 6, pp. 853–860, Dec. 2024.

[2] A. H. Alhilali and A. Montazerolghaem, "An intelligent load balancing strategy to improve performance and QoS in software defined-data center networks," *Far East Journal of Applied Mathematics*, vol. 117, no. 2, pp. 149–167, Sep. 2024.

[3] F. Ullah, M. A. Babar, and T. Hastie, "A systematic literature review for load balancing and task scheduling techniques in cloud computing," *Artificial Intelligence Review*, Springer, vol. 57, no. 10, pp. 1–62, Sep. 2024.

[4] B. Predic, L. Jovanovic, V. Simic, N. Bacanin, M. Zivkovic, P. Spalevic, N. Budimirovic, and M. Dobrojevic, "Cloud-load forecasting via decomposition-aided attention recurrent neural network tuned by modified particle swarm optimization," *Complex Intelligent Systems*, vol. 10, no. 2, pp. 2249–2269, 2024.

[5] J. Santos, T. Wauters, F. De Turck, and P. Steenkiste, "Towards optimal load balancing in multi-zone Kubernetes clusters via reinforcement learning," in *Proc. IEEE Int. Conf. Network and Service Management (CNSM)*, Montreal, QC, Canada, Jul. 2024, pp. 1–7.

[6] C. Redovian, "Meta-reinforcement learning with discrete world models for adaptive load balancing," in *Proc. ACM Southeast Conference (ACMSE 2025)*, Cape Girardeau, MO, USA, Apr. 2025, pp. 1–6.

[7] H. Zheng, K. Xu, M. Zhang, H. Tan, and H. Li, "Efficient resource allocation in cloud computing environments using AI-driven predictive analytics," *Applied and Computational Engineering*, vol. 82, pp. 17–23, 2024.

[8] H. Jian, Q. Zhu, Y. Liu, M. Huang, and X. Guo, "DRS: A deep reinforcement learning enhanced Kubernetes scheduler for microservice-based system," *Software: Practice and Experience*, Wiley, vol. 54, no. 3, pp. 412–430, Oct. 2024.

[9] A. Sharma, V. Balasubramanian, and J. Kamruzzaman, "A temporal deep Q learning for optimal load balancing in software-defined networks," *Sensors*, vol. 24, no. 4, p. 1216, Feb. 2024.

[10] Z. Ahamed, M. Khemakhem, F. Eassa, F. Alsolami, A. Basuhail, and K. Jambi, "Deep reinforcement learning for workload prediction in federated cloud environments," *Sensors*, vol. 23, no. 15, p. 6911, Aug. 2023.

[11] Q. Fettes, A. Karanth, R. Bunescu, B. Beckwith, and S. Subramoney, "Reclaimer: A reinforcement learning approach to dynamic resource allocation for cloud microservices," *arXiv preprint arXiv:2304.07941*, Apr. 2023.

[12] J. Dogani, F. Khunjush, and M. Seydali, "Host load prediction in cloud computing with discrete wavelet transformation and bidirectional gated recurrent unit network," *Complex Intelligent Systems*, Springer, vol. 9, no. 4, pp. 3185–3203, 2023.

[13] T. A. Murshedi, R. M. Almuttairi, R. Satar, G. A. Al-sultany, M. S. Almhanna, F. S. Al-Turaihi, and F. A. ALghurabi, "Optimizing cloud computing: A holistic strategy for efficient and sustainable operation through ant colony load balancing and resource optimization," *International Journal of Scientific Research in Science and Technology*, vol. 17, no. 5, Apr. 2024.

[14] D. Cui, Z. Peng, K. Li, Q. Li, J. He, X. Deng, and S. Naaz, "A novel cloud task scheduling framework using hierarchical deep reinforcement learning for cloud computing," *PLOS ONE*, Aug. 2025.

[15] Y. Sanjalawe, S. Fraihat, S. Al-E'mari, M. Abualhaj, S. Makhadmeh, E. Alzubi, and D. La Torre, "Smart load balancing in cloud computing: Integrating feature selection with advanced deep learning models," *PLOS ONE*, Sep. 2025.

[16] R. N. Calheiros, R. Ranjan, A. Beloglazov, C. A. F. De Rose, and R. Buyya, "CloudSim: A toolkit for modelling and simulation of cloud computing environments and evaluation of resource provisioning algorithms," *Software: Practice and Experience*, vol. 41, no. 1, pp. 23–50, Jan. 2011.

[17] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015.

[18] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, Jul. 2017.

[19] J. Gilliard, M. Hamieh, and A. Montazerolghaem, "Intelligent load balancing in data center software-defined networks," *Transactions on Emerging Telecommunications Technologies*, Wiley, vol. 35, no. 3, e4967, 2024.